Patch-Aware KV Cache Compression for Byte-Level Language Models

A controlled study of attention-guided, entropy-guided and recency eviction

Group 12: Aditya Rayaprolu, Kshitij Gupta, Dhwaj Kishore, Arnab Hazra, Riyazuddin Khan

CS F429 Natural Language Processing, BITS Pilani Hyderabad Campus

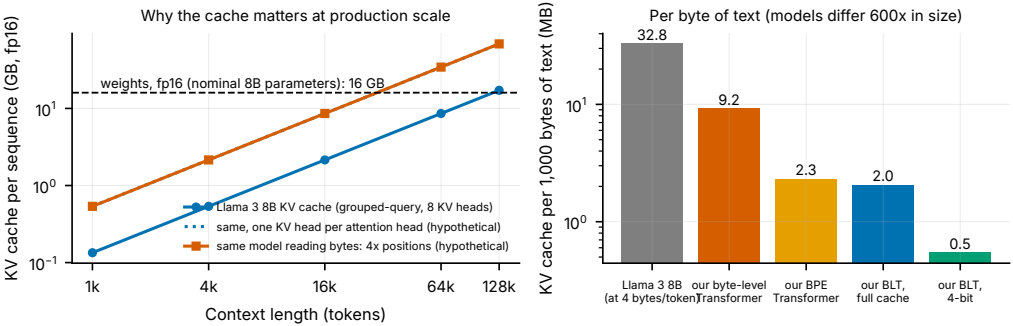
Part 3 presentation

Why this problem

- Chat models read BPE tokens; a tokenizer-free model reads raw bytes
- Bytes: no unknown words, robust to typos, code, other scripts
- Cost: about 4 times more positions, so a 4 times larger KV cache
- The KV cache, not the weights, limits batch size and context length
- BLT already computes next-byte entropy to place patches: free signal?



How it relates to the models people use

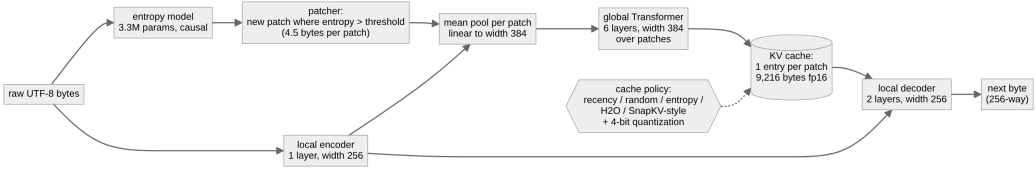


Llama 3 8B: 32 layers, 8 KV heads (grouped-query attention to shrink the cache), 128k tokens; Llama 2 7B: 32k BPE vocabulary, 4k context. Ours: 13.3M parameters, trained from scratch on one GPU. We compare cost structure and methods, not chat quality.

The task and the metric

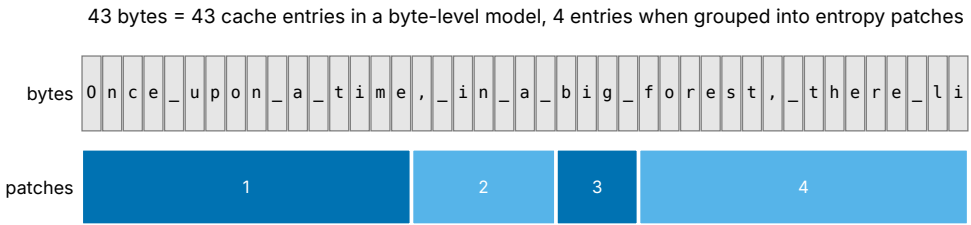
- Task: next-symbol prediction (language modelling), scored in **bits per byte (bpb)**
- BPB is tokenizer-independent: 4-gram, BPE and byte models on identical documents
- Datasets: TinyStories (easy), WikiText-103 (hard, long), The Stack Smol (code mix), PG-19 (books)
- Baselines: Kneser-Ney byte 4-gram (classical), BPE Transformer 13.8M (modern)
- Evaluation: three seeds, paired bootstrap intervals, WikiText test split used once

Our system: a small BLT-style model



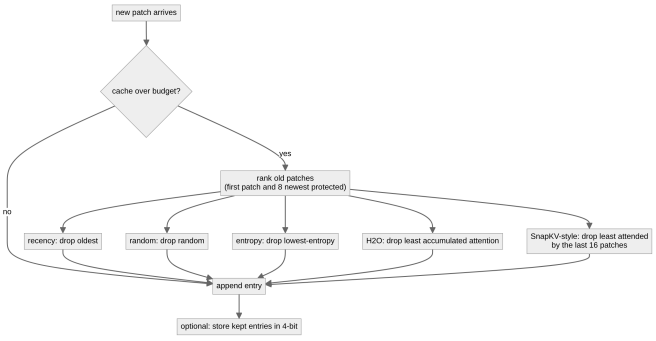
- 13,333,632 parameters; entropy model 3.3M; one cache entry per patch (9,216 bytes fp16)

Patches shrink the cache

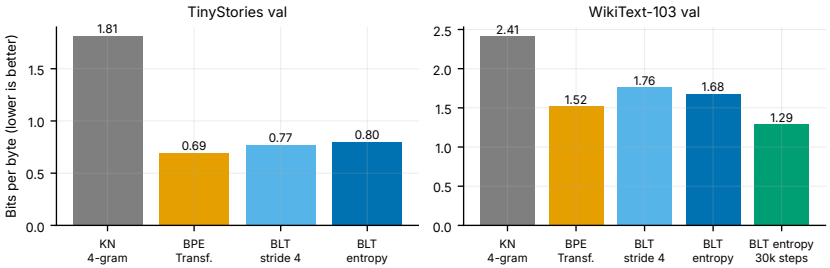


- Patch boundary where the next byte is hard to predict: 67% of patch starts at word starts (chance 16 to 20%)
- Byte model: one entry per byte. BLT-style: one per patch (4.5 bytes)

Cache policies compared at equal cache bytes

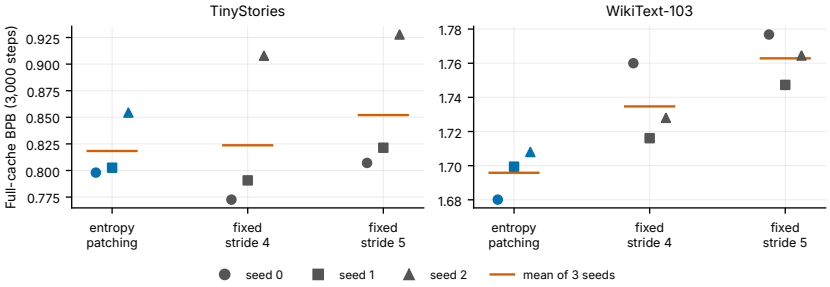


Baselines and base models



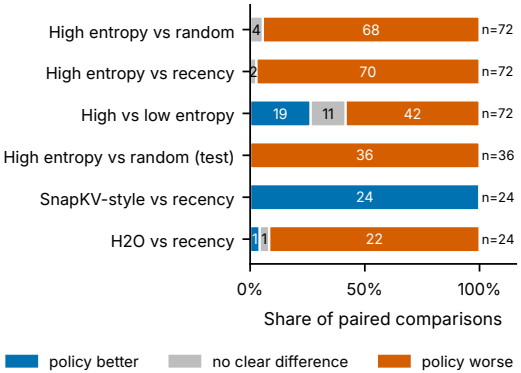
- Every neural model is far ahead of the 4-gram; at equal training the BPE Transformer is ahead of our BLT
- With 10x training the WikiText BLT reaches 1.29 to 1.31 (not like-for-like)

Seeds matter



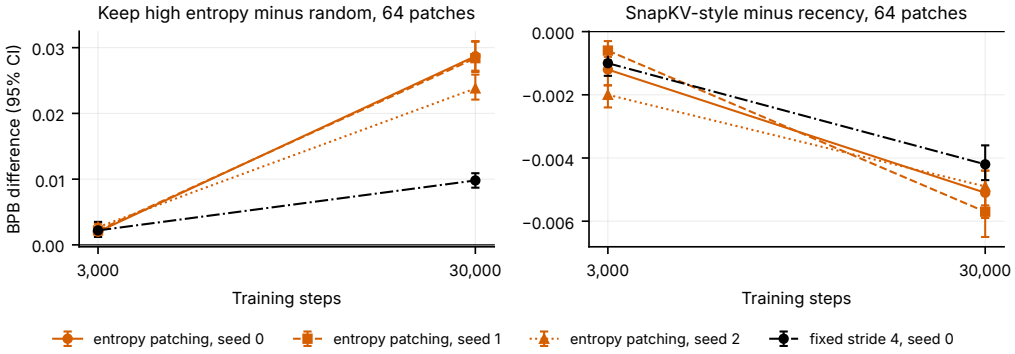
Seed spread (0.03 to 0.14 BPB) is as large as patching gaps: a single-seed 3.3% claim was retired. Entropy patching is lowest on WikiText in 3 of 3 seed pairs (likely, not established).

Main test: is entropy a good keep signal?



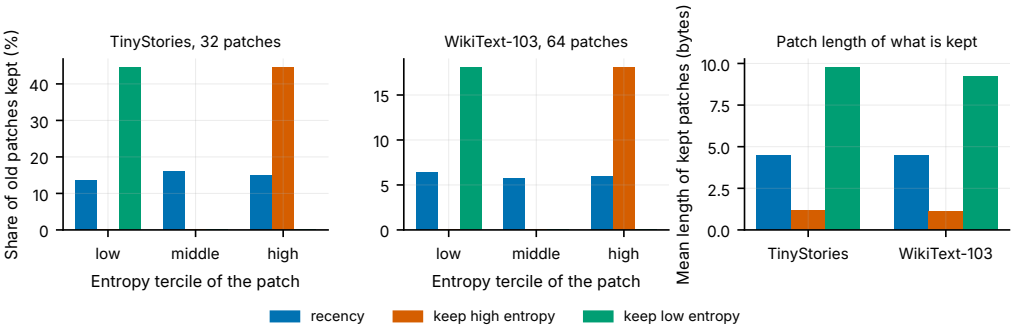
- Worse than random in 68 and than recency in 70 of 72 comparisons, better in none
- Holds in every seed, fixed-stride models, code mix, 16k context, and the test split (36 of 36)
- SnapKV-style (online): better than recency in 24 of 24

More training makes it worse, not better



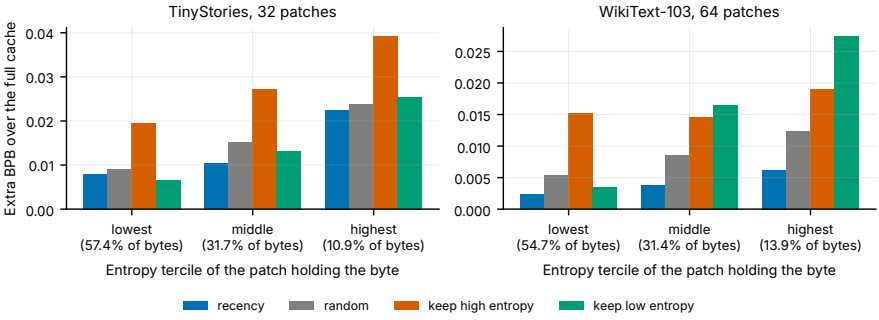
Penalty against random at 64 patches: +0.002 (3,000 steps) to +0.024 to +0.029 (30,000 steps). SnapKV-style gain grows several times as well.

Why: what the entropy rule keeps



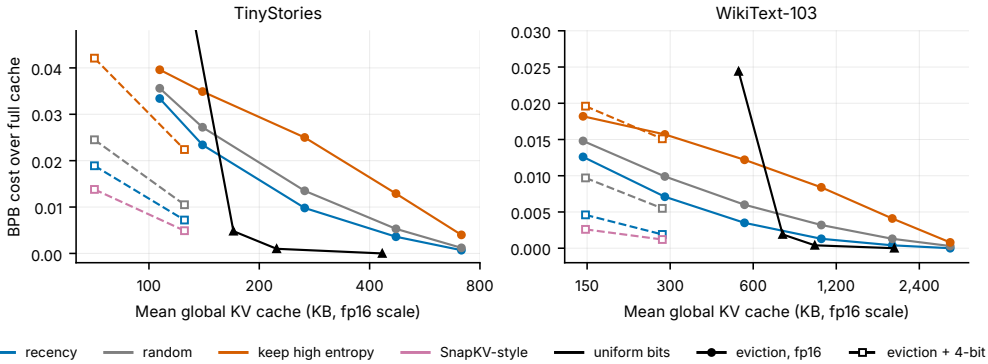
- It keeps one- and two-byte fragments (98.0% and 99.7% of kept patches) and drops patches with words
- Old high-entropy patches also receive less attention in middle and upper layers

Where the extra loss falls



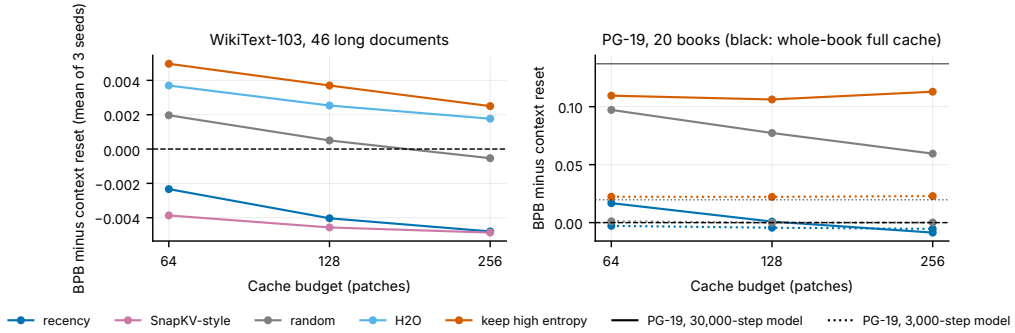
Keeping high-entropy patches does not protect the high-entropy bytes and also costs the easy ones. The top 1% of bytes carry 18 to 30% of the extra loss.

What works: eviction, quantization, SnapKV-style



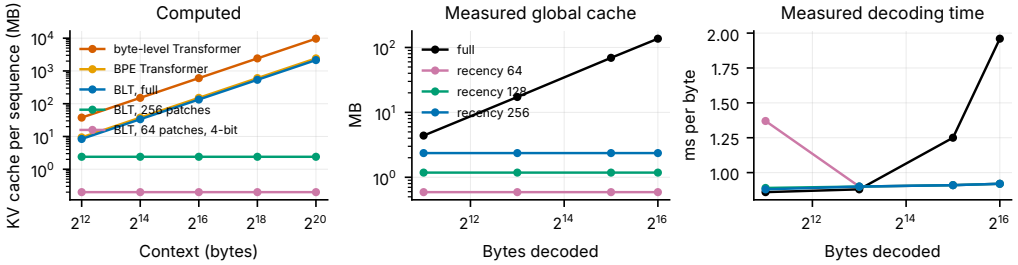
WikiText: 128 patches in 4-bit keep 7.4% of the cache bytes for $+0.0012$ BPB (SnapKV-style); keeping high entropy never reaches the frontier.

Bounded-memory streaming



A fixed recency cache beats resetting the context in 3 of 3 seeds and on 20 PG-19 books (by 0.0086 BPB).

Measured memory and time



0.59 MB against 136.5 MB at 65,536 bytes; 0.9 ms per byte against a rise to 2.0 ms.

Compute per byte (analytic)

Model	1,024 bytes MFLOPs	MB	4,096 bytes MFLOPs	MB
BLT 13.3M, full global cache	11.06	2.1	12.46	8.4
BLT, 64-patch cache	10.73	0.6	10.73	0.6
BLT, full cache, plus entropy model	19.58	2.1	20.98	8.4
BLT, 64-patch cache, plus entropy model	19.25	0.6	19.25	0.6
BPE transformer 13.8M, full cache	7.43	2.4	9.20	9.4
BPE transformer, 64-token cache	6.99	0.6	6.99	0.6
Byte-level transformer, full cache	30.87	9.4	59.18	37.7
Byte-level transformer, 64-entry cache	22.02	0.6	22.02	0.6
Byte-level transformer, 288-entry cache (same text span as 64 patches)	24.08	2.7	24.08	2.7

- BLT: 2.8x fewer FLOPs than a byte Transformer
- But more than BPE (11.1 against 7.4), and the entropy model adds 8.5
- Case for the BLT: cache memory

Error analysis: generations diverge

TinyStories, prompt of 700 bytes, 32 patches (real output, greedy):

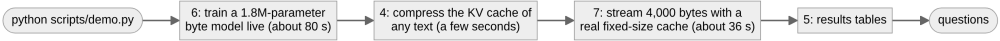
- full: `find your friends." Roxy was so happy to help Roxy.`
- recency (diverges at byte 17): `find your friends to climb the leaves."...`
- keep high entropy (byte 25): `...Roxyy said, "Thank you, Billy.`

Per-byte: `They argue[d]` costs 0.02 bits with the full cache and 5.97 under entropy eviction. Small models repeat; these are anecdotes of where the compressed cache changes the prediction.

Challenges and how we handled them

- Seed variance as large as gaps: three seeds, paired bootstrap; a 3.3% claim retired
- Byte-share accounting error found by cross-table check: fixed, all tables regenerated
- Under-trained 16k models gave a wrong conclusion: tuned, penalty does not fade
- Offline attention scores approximate: online implementation built and tested
- Silent OOM corrupted a cache: caught from logs, queue hardened
- Gated code data, PG-19 loader, cluster limits: workarounds recorded in `CHALLENGES.md`

Live demo (CPU, no GPU needed)



`python scripts/demo.py` (or `run.bat`); script and fallbacks in `docs/demo_script.md`

Scope, limits and what is next

- 13.3M parameters; the models do not retrieve (planted-name test): retrieval tasks untested
- Cache sizes computed except recency; FLOPs analytic; PG-19 confounds entropy with distance
- Next: larger models, positions renumbered inside the cache, attention policies in step decoding

Conclusion

- **New:** controlled test of entropy-ranked cache retention on a byte-level patch model; online attention policies over patch caches; measured bounded-memory streaming
- **Improved:** 7.4% of the cache for $+0.0012$ BPB; SnapKV-style beats recency 24/24; constant memory and time; 2.8x fewer FLOPs than a byte Transformer
- **Did not work:** entropy as the keep signal (68 to 70 of 72 worse), and it grows worse with training
- **Open:** larger models and retrieval